SecondBrain - RAG + MCP Smart Notes Assistant

Final-Year CSE Project - Complete Build Context

> ***Purpose of this document:** This is the single source of truth for building the project. Hand this entire file to an AI coding IDE (e.g. Antigravity / Claude Code) and ask it to scaffold and implement the project step by step. Every architectural decision, file, tool, and deployment step needed is described here.*

---

1. Project Summary

SecondBrain is a personal knowledge assistant that:

1. **Reads** your Markdown notes using **RAG (Retrieval-Augmented Generation)** - it embeds your notes into a vector database and answers questions grounded in *your* content, with citations.
2. **Acts** on your notes using an **MCP (Model Context Protocol) server** - it can create notes, add `[[wiki-links]]`, suggest tags, and find orphan notes.

The novelty (and the one-line viva pitch):

> **"Retrieval grounds the answers; the MCP server lets the agent act on the knowledge base - so it is an active second brain, not a read-only chatbot. It is a bidirectional read/write loop."**

Target user: A student/developer who keeps notes in Markdown and wants to chat with them AND let an AI help organize them.

Cost constraint: Rs.0. Everything uses free/open-source tools and free hosting tiers.

Author hardware: Windows 11 laptop, 16 GB RAM (can run a 3B-8B local model comfortably).

---

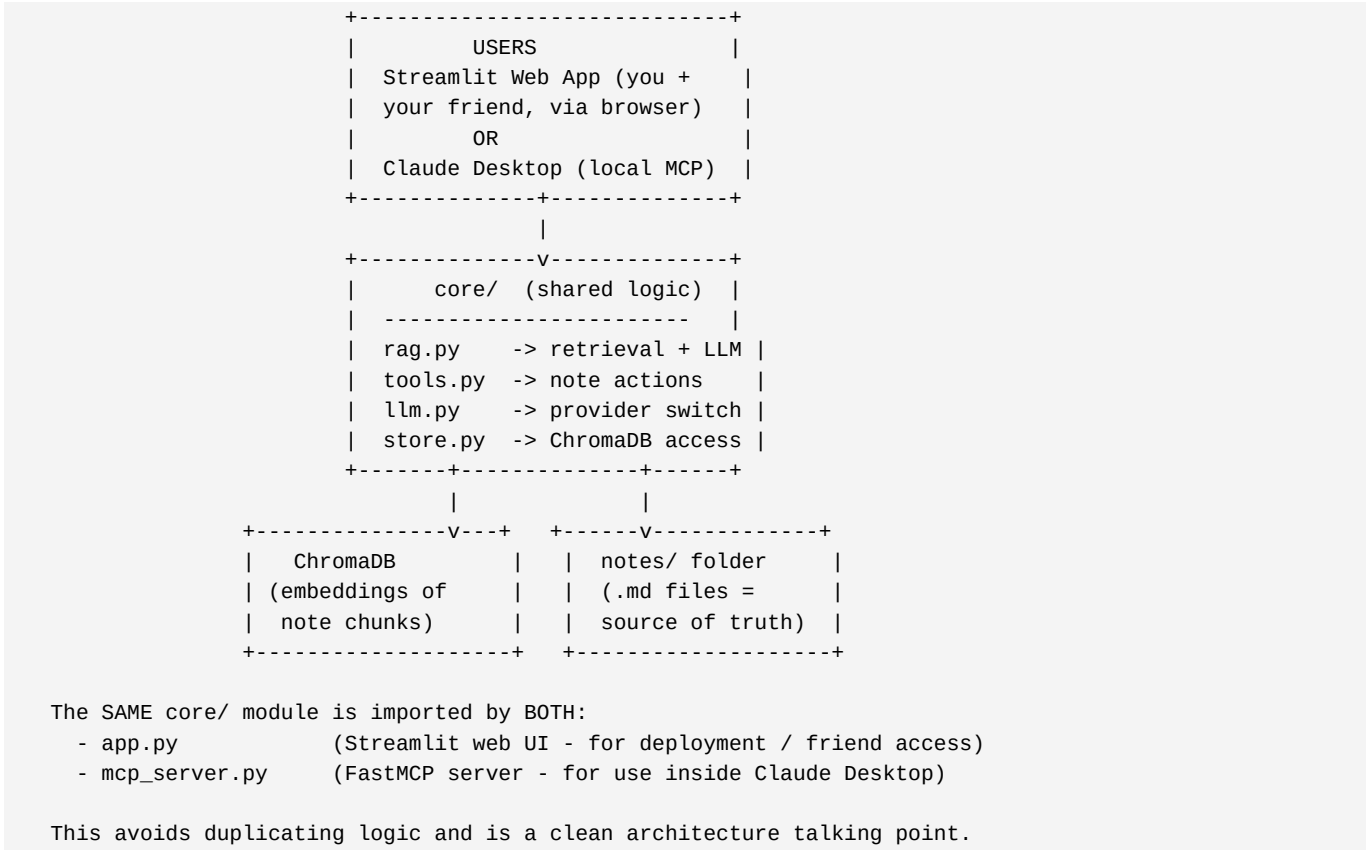
2. Tech Stack (all free)

Layer	Technology	Notes
Language	**Python 3.11**	Already installed
LLM (local dev)	**Ollama** + `llama3.2:3b` (or `phi3:mini`)	Fully offline, free
LLM (deployed)	**Groq API** (free tier) - `llama-3.1-8b-instant`	Free API key, fast, for remote friend access
Embeddings	**sentence-transformers** `all-MiniLM-L6-v2`	Local, CPU, free, ~80MB
Vector DB	**ChromaDB** (persistent, local file)	Free, no server needed
MCP server	**FastMCP** (`mcp` Python SDK)	Open standard, free
Web UI	**Streamlit**	Free, fast to build
Deployment	**Streamlit Community Cloud**	Free public hosting, friend-accessible
Versioning	**Git + GitHub**	Free

Key design rule: The LLM provider is **swappable via an environment variable** (`LLM_PROVIDER=ollama` or `LLM_PROVIDER=groq`). Local development uses Ollama (offline); the deployed version uses Groq (because Ollama cannot be hosted for free for a remote friend to reach).

---

3. Architecture



Why a shared `core/` module? MCP servers normally talk to a desktop client over stdio. A deployed web app can't use that transport. So we put all real logic in `core/`, and expose it two ways: (a) as MCP tools for Claude Desktop, and (b) directly inside the Streamlit app for the deployed/friend-facing version. One brain, two faces.

---

4. Features

MVP (build these first - this alone is a complete, gradeable project)

1. **Ingest** a folder of `.md` notes -> chunk by heading -> embed -> store in ChromaDB.
2. **Ask** - semantic Q&A over notes with **citations** (which note + heading the answer came from).
3. **Create note** - "summarize these into a new note" -> writes a new `.md` file.
4. **Suggest tags** - read a note, propose tags drawn from the existing tag vocabulary.

Bonus (add if time allows - also great "Future Work" if not)

5. **Auto-linking** - suggest `[[wiki-links]]` between semantically related notes (cosine similarity).
6. **Orphan detection** - list notes that nothing links to.
7. **Daily digest** - summarize notes created/edited today.
8. **Link graph** - simple visualization of note connections.

---

5. Folder Structure

```
SecondBrain/
+-- README.md
+-- requirements.txt
+-- .env.example                # template for env vars (no secrets committed)
+-- .gitignore
+-- notes/                      # the Markdown vault (sample notes for demo)
|   +-- welcome.md
|   +-- rag-basics.md
|   +-- mcp-basics.md
+-- core/
|   +-- __init__.py
|   +-- config.py               # loads env vars, paths, model names
|   +-- llm.py                  # provider switch: ollama | groq
|   +-- store.py                # ChromaDB client + collection helpers
|   +-- ingest.py               # markdown -> chunks -> embeddings -> ChromaDB
|   +-- rag.py                  # retrieve top-k + build prompt + call LLM
|   +-- tools.py                # note actions: search, create, suggest_tags, links, orphans
+-- mcp_server.py               # FastMCP server exposing core/tools.py as MCP tools
+-- app.py                      # Streamlit web UI (deployed version)
+-- scripts/
|   +-- reindex.py              # CLI: rebuild the whole index from notes/
+-- docs/
    +-- PROJECT_CONTEXT.md      # this file
    +-- REPORT_OUTLINE.md       # thesis chapter outline
```

---

6. Component Specifications

6.1 `core/config.py`

- Loads `.env` (use `python-dotenv`).
- Exposes: `NOTES_DIR`, `CHROMA_DIR`, `EMBED_MODEL` (`all-MiniLM-L6-v2`), `LLM_PROVIDER`, `OLLAMA_MODEL`, `GROQ_MODEL`, `GROQ_API_KEY`, `TOP_K` (default 4), `CHUNK_MAX_CHARS` (default 1200).

6.2 `core/llm.py`

- Single function `chat(system: str, user: str) -> str`.
- If `LLM_PROVIDER == "ollama"`: call local Ollama via the `ollama` python package (`ollama.chat(model=OLLAMA_MODEL, messages=[...])`).
- If `LLM_PROVIDER == "groq"`: call Groq via the `groq` python package (OpenAI-compatible).
- This is the ONLY file that knows about a specific LLM vendor. Everything else calls `chat()`.

6.3 `core/store.py`

- Create a **persistent** ChromaDB client at `CHROMA_DIR`.
- One collection: `notes`.
- Helper functions: `get_collection()`, `add_chunks(ids, texts, metadatas, embeddings)`, `query(embedding, k)`, `reset_collection()`.
- Use sentence-transformers to embed (do NOT use Chroma's default embedder - we control it, so dev and deploy behave identically).

6.4 `core/ingest.py`

- Walk `NOTES_DIR` for `*.md`.
- **Chunk by Markdown heading** (split on `#`, `##`, `###`). If a section is larger than

`CHUNK_MAX_CHARS`, sub-split on paragraph boundaries.

- Each chunk's metadata: `{ "source": <relative path>, "heading": <heading text>, "note_title": <H1 or filename> }`.
- Embed all chunks with `all-MiniLM-L6-v2`, write to ChromaDB.
- Idempotent: `reset_collection()` then re-add (simple and correct for a student project).

6.5 `core/rag.py`

- `answer(question: str) -> dict`:
1. Embed the question.
2. Retrieve top-`TOP_K` chunks from ChromaDB.
3. Build a prompt: system = "Answer ONLY from the provided context. Cite sources as [note_title > heading]. If the answer isn't in the context, say so."
4. Call `llm.chat()`.
5. Return `{ "answer": str, "sources": [ {source, heading, note_title} ] }`.
- This grounding + citation behavior is the core RAG deliverable. Emphasize it in the report.

6.6 `core/tools.py` - the note-action functions (these become MCP tools)

Function	Signature	Behavior
`search_notes`	`(query: str, k: int=4) -> list[dict]`	Semantic search - the RAG<->MCP bridge
`read_note`	`(path: str) -> str`	Return full file text
`list_notes`	`() -> list[str]`	All note paths
`create_note`	`(title: str, body: str, tags: list[str]=[]) -> str`	Write a new `.md` with YAML front-matter; re-index it; return path
`append_to_note`	`(path: str, text: str) -> str`	Append text; re-index
`suggest_tags`	`(path: str) -> list[str]`	LLM proposes tags from existing tag vocabulary
`add_link`	`(from_path: str, to_title: str) -> str`	Insert a `[[to_title]]` wiki-link
`find_orphans`	`() -> list[str]`	Notes with no inbound `[[links]]`

> ***Safety:** `create_note`/`append_to_note`/`add_link` must only write **inside** `NOTES_DIR` (validate the resolved path is within the vault - prevents path traversal). Mention this as a security consideration in the report.*

6.7 `mcp_server.py`

- Use **FastMCP**: `from mcp.server.fastmcp import FastMCP`.
- `mcp = FastMCP("SecondBrain")`.
- Decorate each function from `core/tools.py` with `@mcp.tool()` (thin wrappers).
- Run with stdio transport for Claude Desktop.
- Provide the Claude Desktop config snippet in the README (see §8.3).

6.8 `app.py` - Streamlit UI (deployed, friend-facing)

Pages/sections:

- **Ask** - text box -> calls `rag.answer()` -> shows answer + expandable "Sources".
- **Notes** - list notes, view a note, run "Suggest tags" / "Find orphans".
- **Create** - form (title, body) -> `tools.create_note()`.
- **Upload** - let a user upload `.md` files into `notes/` then trigger re-index
(so your friend can use their own notes).
- Sidebar shows which `LLM_PROVIDER` is active and a "Re-index notes" button.

---

7. Implementation Roadmap (build order for the AI IDE)

Build and test in this exact order. Do not start a step until the previous one runs.

1. **Scaffold** the folder structure + `requirements.txt` + `.env.example` + `.gitignore` + 3 sample notes.
2. **`core/config.py`** - env loading. Verify it prints config.
3. **`core/store.py`** + **`core/ingest.py`** - index the 3 sample notes. Verify ChromaDB has chunks.
4. **`core/llm.py`** - wire up Ollama first. Test `chat("be brief","say hi")`.
5. **`core/rag.py`** - full Q&A with citations on the sample notes. This is the MVP heartbeat.
6. **`core/tools.py`** - `search_notes`, `create_note`, `suggest_tags`. Test each as a plain function.
7. **`mcp_server.py`** - wrap tools; test in Claude Desktop.
8. **`app.py`** - Streamlit UI hitting `core/`. Run locally (`streamlit run app.py`).
9. **Swap test** - set `LLM_PROVIDER=groq`, confirm the app still works with a Groq key.
10. **Bonus features** - auto-link, orphans, daily digest.
11. **Deploy** to Streamlit Community Cloud (see §8.4).
12. **Write report** using `docs/REPORT_OUTLINE.md` and the evaluation in §9.

---

8. Setup & Run Instructions

8.1 Local environment

```
python -m venv .venv
.venv\Scripts\activate            # Windows PowerShell
pip install -r requirements.txt
copy .env.example .env            # then edit .env
```

8.2 Install & run Ollama (free, offline - for development)

```
# Download Ollama from https://ollama.com (free), then:
ollama pull llama3.2:3b
# Ollama runs a local server automatically on http://localhost:11434
```

Set in `.env`:

```
LLM_PROVIDER=ollama
OLLAMA_MODEL=llama3.2:3b
```

8.3 Index notes and run

```
python scripts/reindex.py        # builds ChromaDB from notes/
streamlit run app.py             # open http://localhost:8501
```

For Claude Desktop (MCP mode), add to its config:

```
{
  "mcpServers": {
    "secondbrain": {
      "command": "python",
      "args": ["C:\\Users\\Admin\\SecondBrain\\mcp_server.py"]
    }
  }
}
```

8.4 Deploy free (so your friend can use it)

1. Push the repo to **GitHub** (public).
2. Get a **free Groq API key** at https://console.groq.com (no card needed).
3. Go to **share.streamlit.io**, connect the repo, set the main file to `app.py`.
4. In Streamlit Cloud **Secrets**, add:

```
LLM_PROVIDER="groq"
GROQ_API_KEY="gsk_...."
GROQ_MODEL="llama-3.1-8b-instant"
```

5. Deploy -> you get a public `https://yourapp.streamlit.app` URL to share with your friend.

***Why Groq for deployment?** Streamlit's free tier can't run Ollama. Groq's free tier serves Llama models over an API at no cost - perfect for a shared, friend-accessible demo. The embedding model still runs inside the app (CPU is fine for `all-MiniLM-L6-v2`).*

***Deployment note on notes storage:** Streamlit Cloud has an ephemeral filesystem (resets on restart). For the demo, either (a) commit sample notes into the repo, or (b) use the Upload page so users add notes per-session. State this limitation honestly in the report - and list "persistent storage (e.g. Supabase free tier)" as Future Work.*

---

9. Evaluation Plan (examiners will ask "how did you measure it?")

Build a small ground-truth set of ~30 questions whose answers you know from the notes. Then report:

Metric	How to measure
Retrieval hit-rate@k	For each question, is the correct note in the top-k retrieved? (k=3 and k=5)
Answer correctness	Manually score answers (correct / partial / wrong)
Citation accuracy	Does the cited source actually contain the answer?
Tag-suggestion precision	Of suggested tags, how many you'd accept
Latency	Avg seconds per query (Ollama vs Groq comparison - nice table)

A clean before/after or Ollama-vs-Groq comparison table makes a strong Results chapter.

---

10. requirements.txt (target contents)

```
streamlit
chromadb
sentence-transformers
python-dotenv
ollama
groq
mcp
markdown-it-py
```

---

11. Viva / Defense Cheat-Sheet

- **What is RAG?** Retrieval + generation: embed docs, fetch relevant chunks, feed to LLM so answers are grounded in my data, not the model's memory. Reduces hallucination, enables citations.
- **What is MCP?** An open standard that lets an LLM call external tools/data through a uniform interface. I built an MCP server exposing note actions (create, link, tag).

- **Where do they meet?** My `search_notes` MCP tool runs the vector retrieval - so retrieval is exposed as a tool, and the agent both reads (RAG) and writes (MCP) the knowledge base.
- **Why these tools?** All free and offline-capable; runs on a 16GB laptop; deployable on a free tier.
- **What's novel?** The bidirectional read/write loop and exposing retrieval itself as an MCP tool.
- **Limitations / Future Work?** Persistent multi-user storage, better chunking, hybrid (keyword+vector) search, evaluation on a larger corpus, auth for the deployed app.

End of project context. Build in the order of §7. Start with the MVP in §4.